

Lecture 4

The Application Layer

Part 2

Subjects of today:

- The Domain Name System
- Socket programming revisited
- Lab work:
 - Wireshark Lab
 - Python Lab 1 + 2

4.1 The Domain Name System

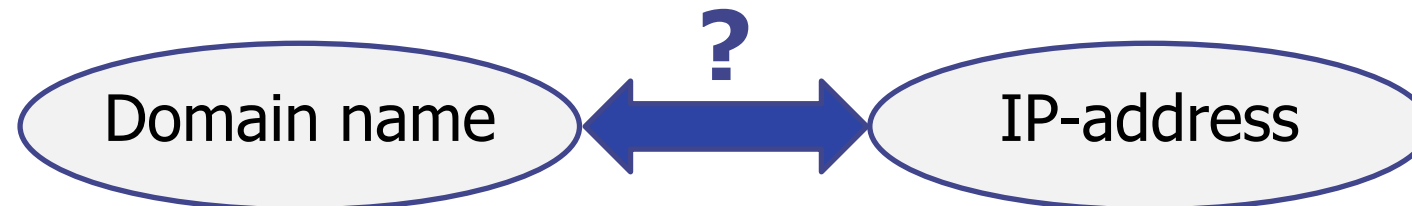
DNS: domain name system

People: many identifiers:

- SSN, name, passport#

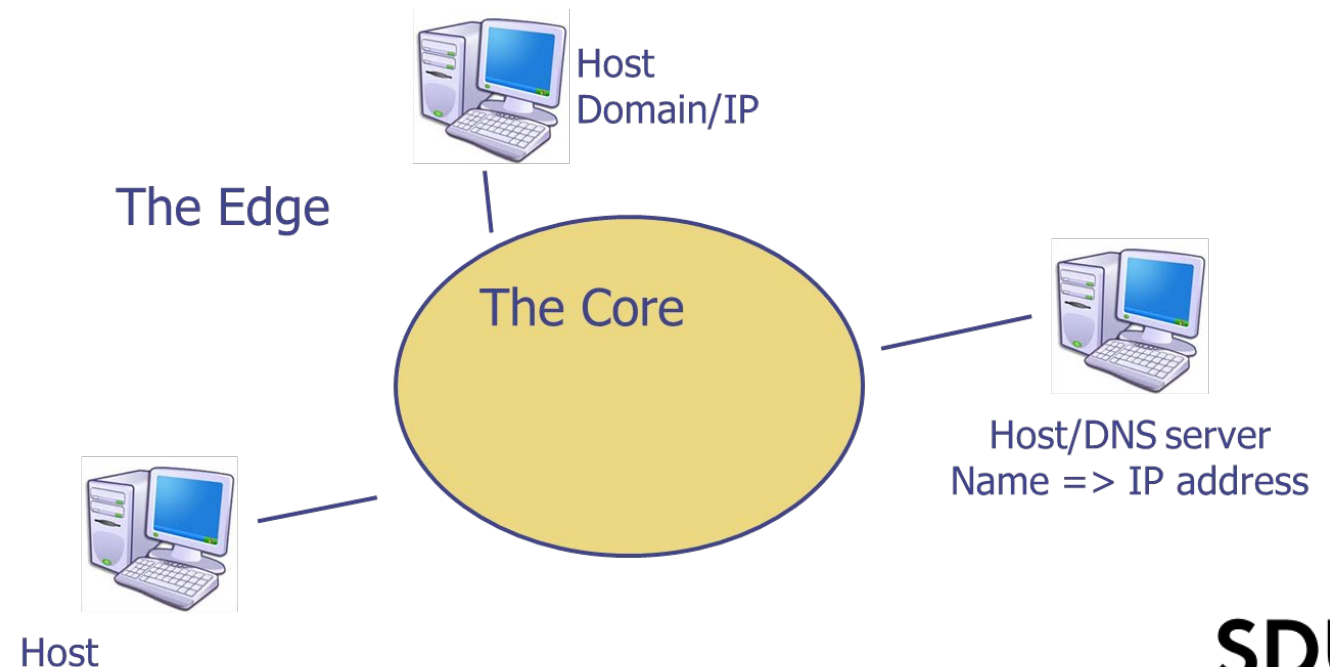
Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- "name", e.g., cs.umass.edu - used by humans



DNS: domain name system

- Distributed database implemented in hierarchy of many name servers
- *Application-layer protocol*: Hosts, name servers communicate to resolve names
 - Note: Core Internet function, **implemented as application-layer protocol**
 - Complexity at network's "edge"

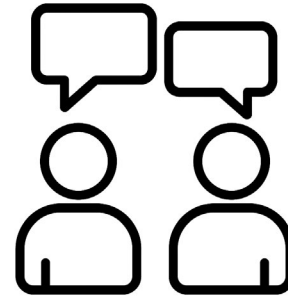


DNS: services, structure

- Hostname to IP address translation
- Host aliasing  e.g. gmail.com & googlemail.com
 - Many hostnames to one IP
 - Servers often have more “specific” canonical hostnames (due to infrastructure)
- Mail server aliasing
 - Mail server can have same name as a webserver
- Load distribution
 - Many IP addresses correspond to one hostname

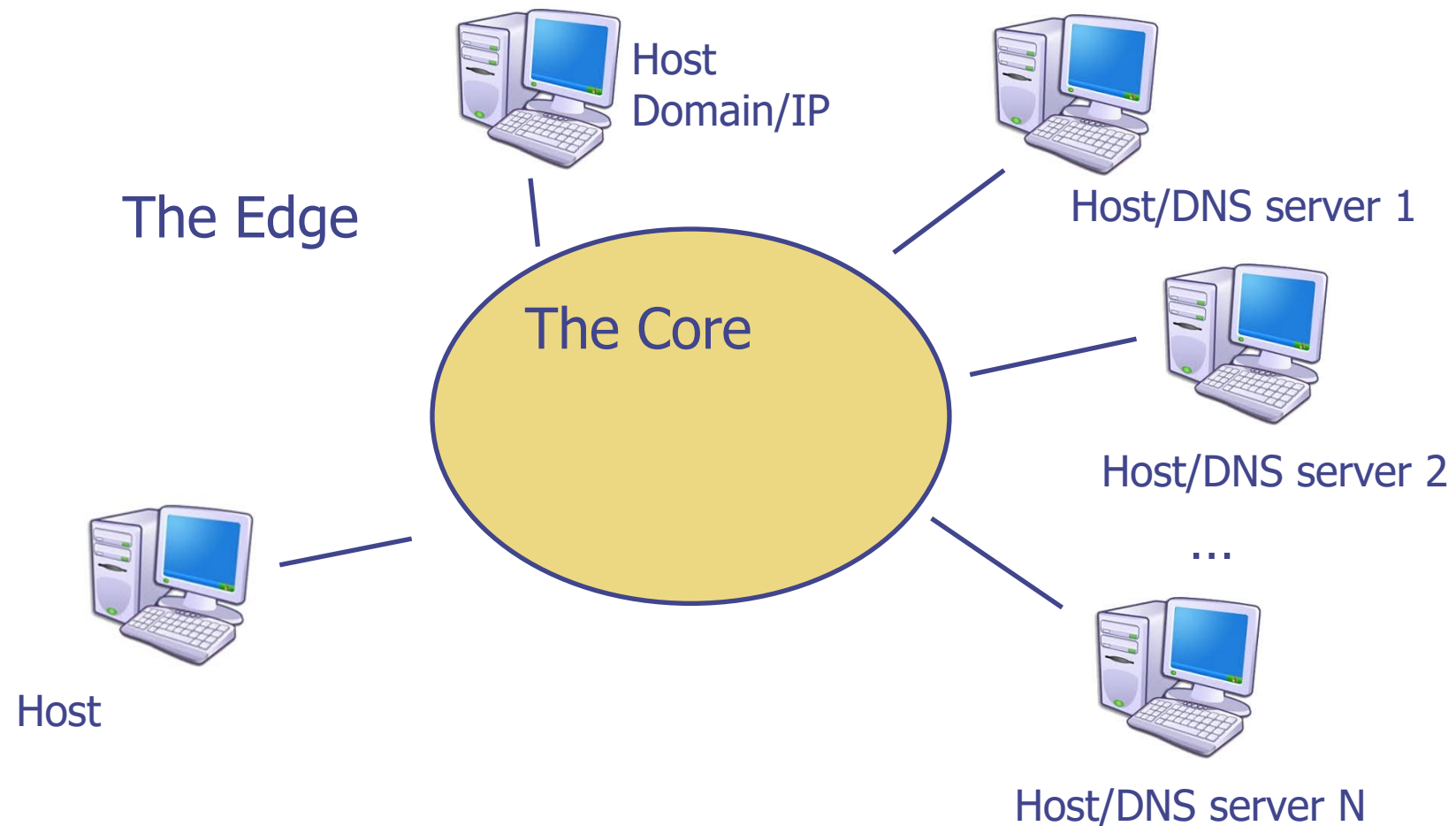
Centralized DNS?

- Single point of failure
- Traffic volume
- Distant centralized database
- Maintenance



Should
implementation be
centralized or
decentralized? Why?

DNS - An application at the edge of the internet.

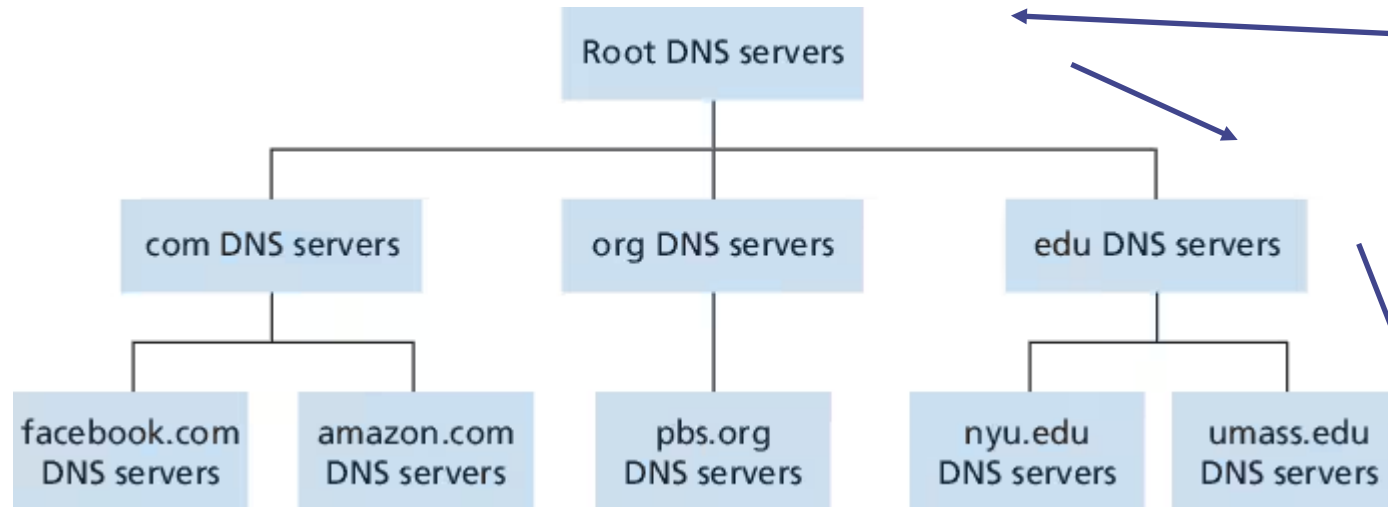


DNS: a distributed, hierarchical database



Top-level:

Authoritative:



client wants IP for www.amazon.com; 1st approximation:

- client queries root server to find com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

- Root name server:
 - Contacts authoritative name server if name mapping not known
 - Gets mapping
 - Returns mapping to local name server



*13 logical root name “servers”
worldwide; each “server,” replicated
many times (around 1000)*

Top-Level Domain, Authoritative servers

TLD servers:

- Responsible for com, org, net, edu, aero, jobs, museums, and all top-level country domains, e.g.: uk, fr, ca, jp
- *Network Solutions* maintains servers for .com TLD
- *Educause* for .edu TLD
- *Punktum dk* for .dk

Authoritative DNS servers:

- Has the final authority on the name resolution!
- Organization's own DNS server(s), providing authoritative hostname/IP mappings for organization's named hosts
- Can be maintained by organization or service provider

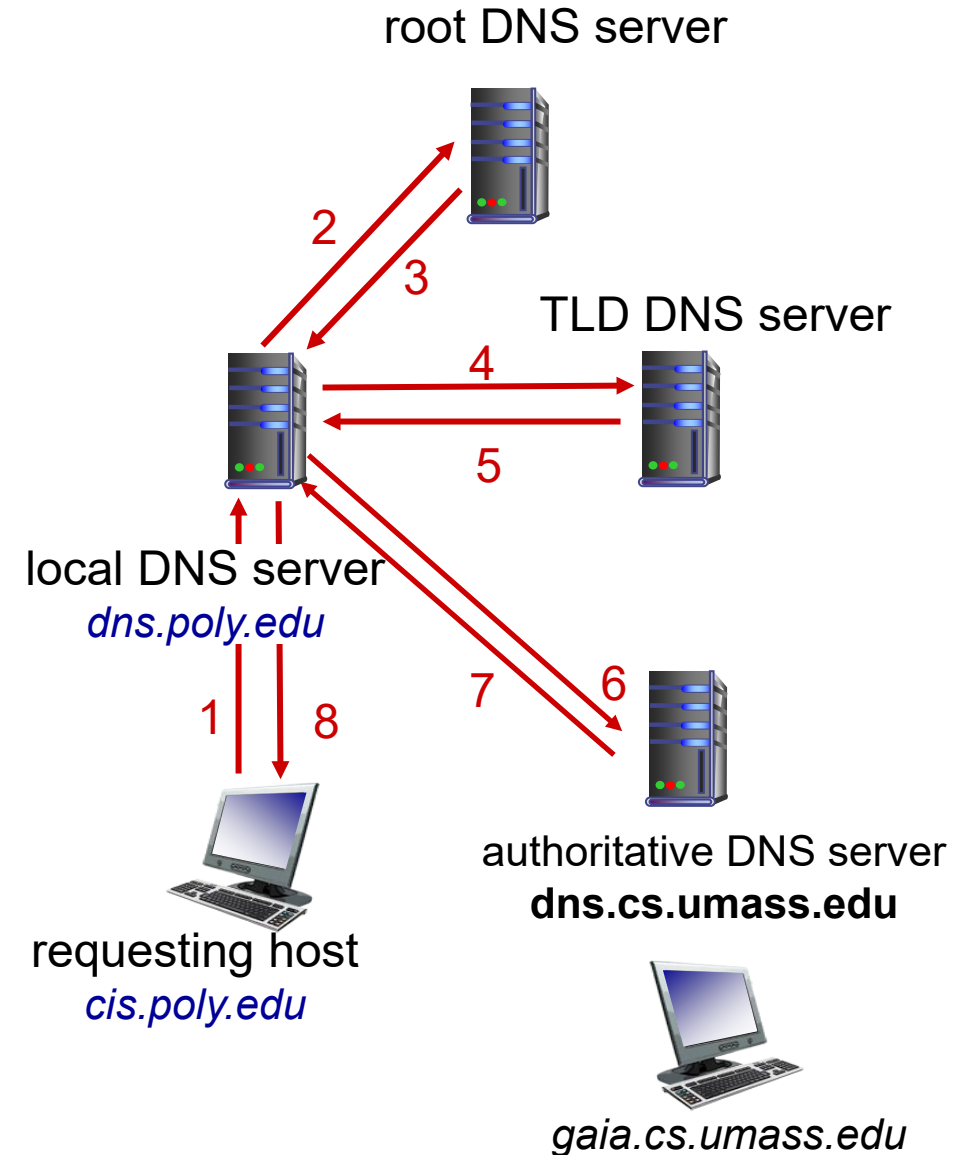
Local DNS name server

- Does not strictly belong to hierarchy
- Each ISP (residential ISP, company, university) has one
 - Also called “default name server”
- When host makes DNS query, query is sent to its local DNS server
 - Acts as proxy, forwards query into hierarchy

DNS name resolution example

Iterative query:

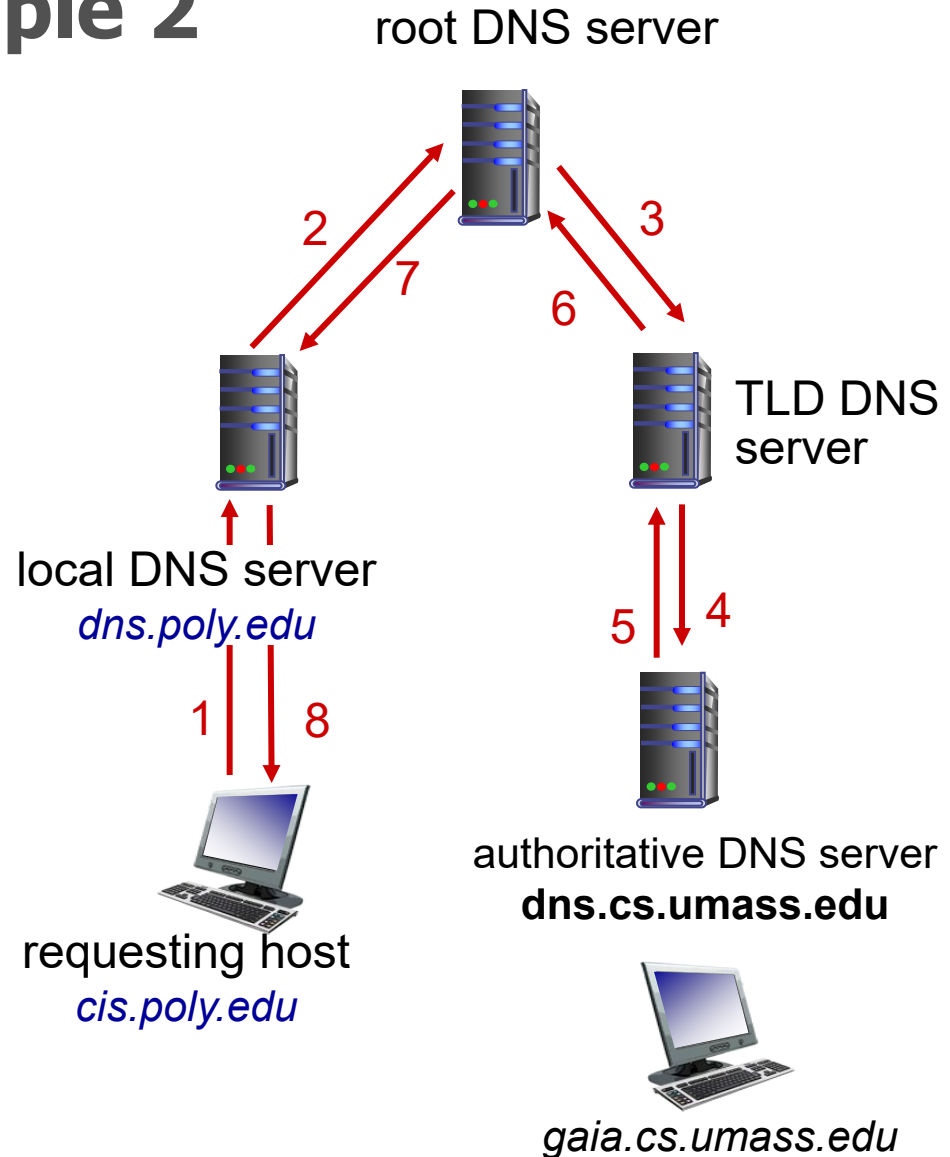
- Contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”



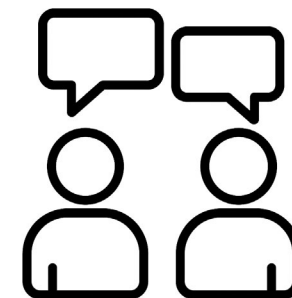
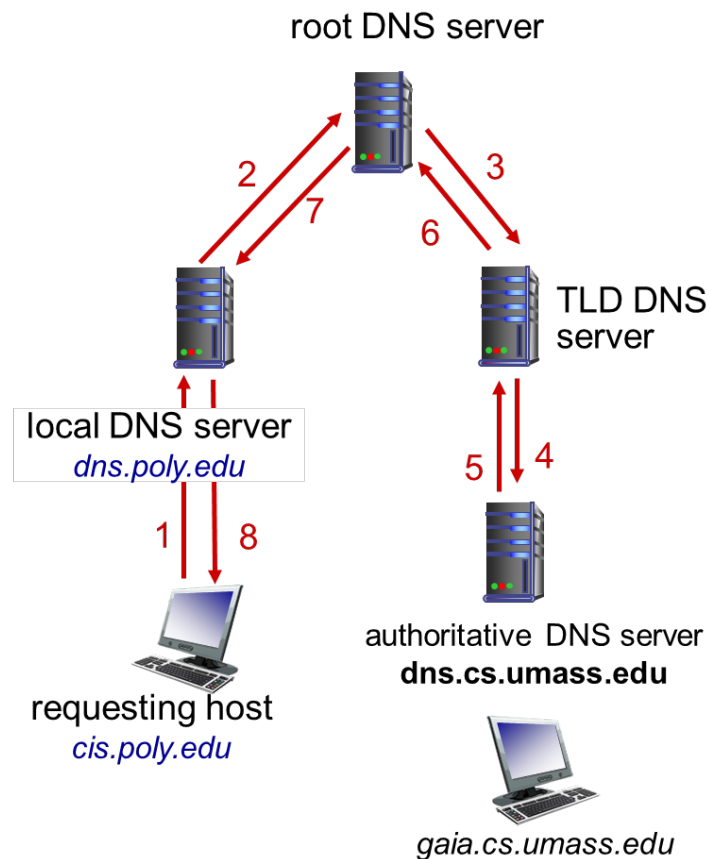
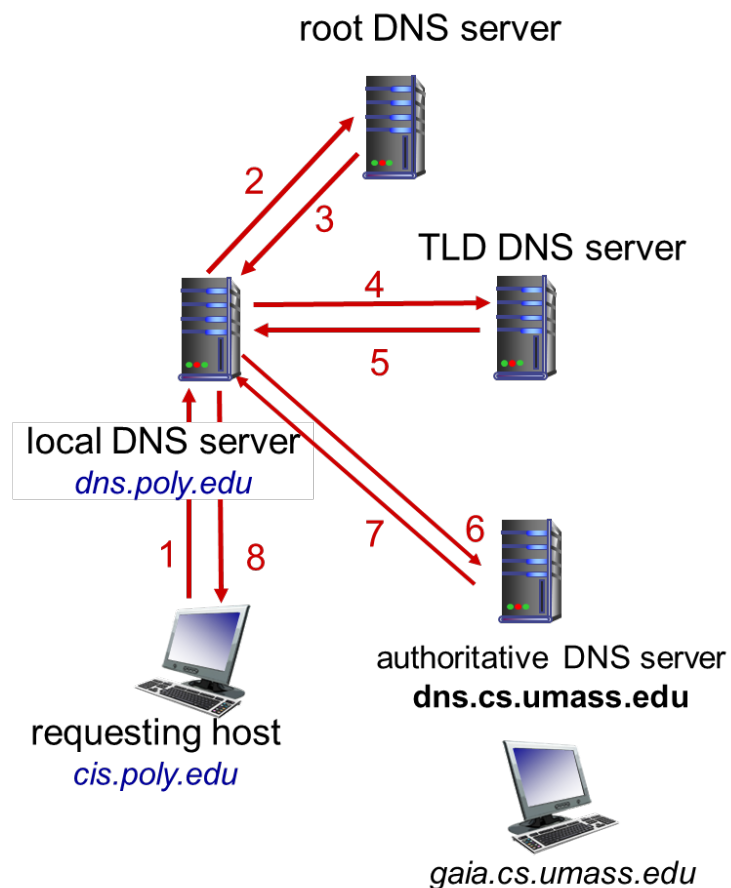
DNS name resolution example 2

Recursive query:

- Requesting host makes a query for a full name resolution



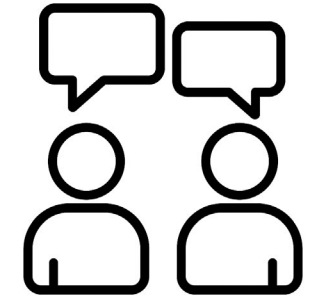
DNS name resolving resource considerations



Discuss where are the
load heaviest in the
two scenarios?

DNS: caching, updating records

- Once (any) name server learns a mapping, it *caches* that mapping
 - Cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
 - Thus, root name servers not often visited
- Cached entries may be *out-of-date*
 - If name host changes IP address, may not be known Internet-wide until all TTLs expire
 - **Local DNS servers typically have a cache (for speed!)**
- Update/notify mechanisms defined in RFC 2136



What could be arguments for not using automatic updates?

DNS records

DNS: Distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- **name** is hostname
- **value** is IP address

type=CNAME

- **name** is alias name for some canonical name
- **value** is canonical name
- Recall: **www.ibm.com** is really **servereast.backup2.ibm.com**

type=NS

- **name** is domain (e.g., foo.com)
- **value** is hostname of authoritative name server for this domain

type=MX

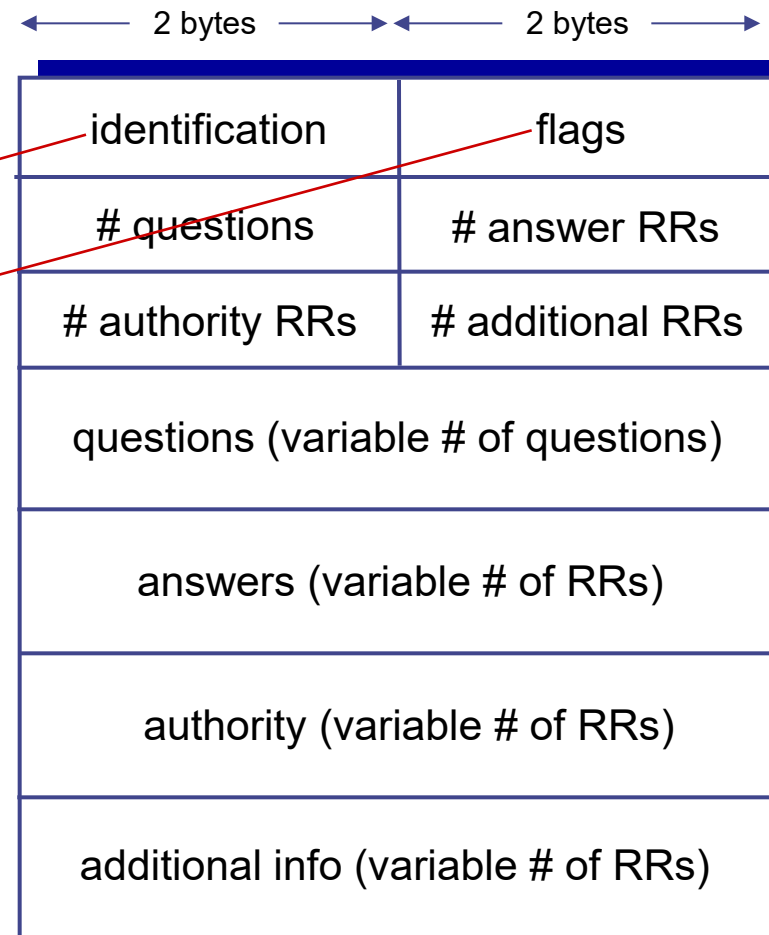
- **value** is name of mailserver associated with **name**
- Recall: Mailserver can have same as a webserver

DNS protocol, messages

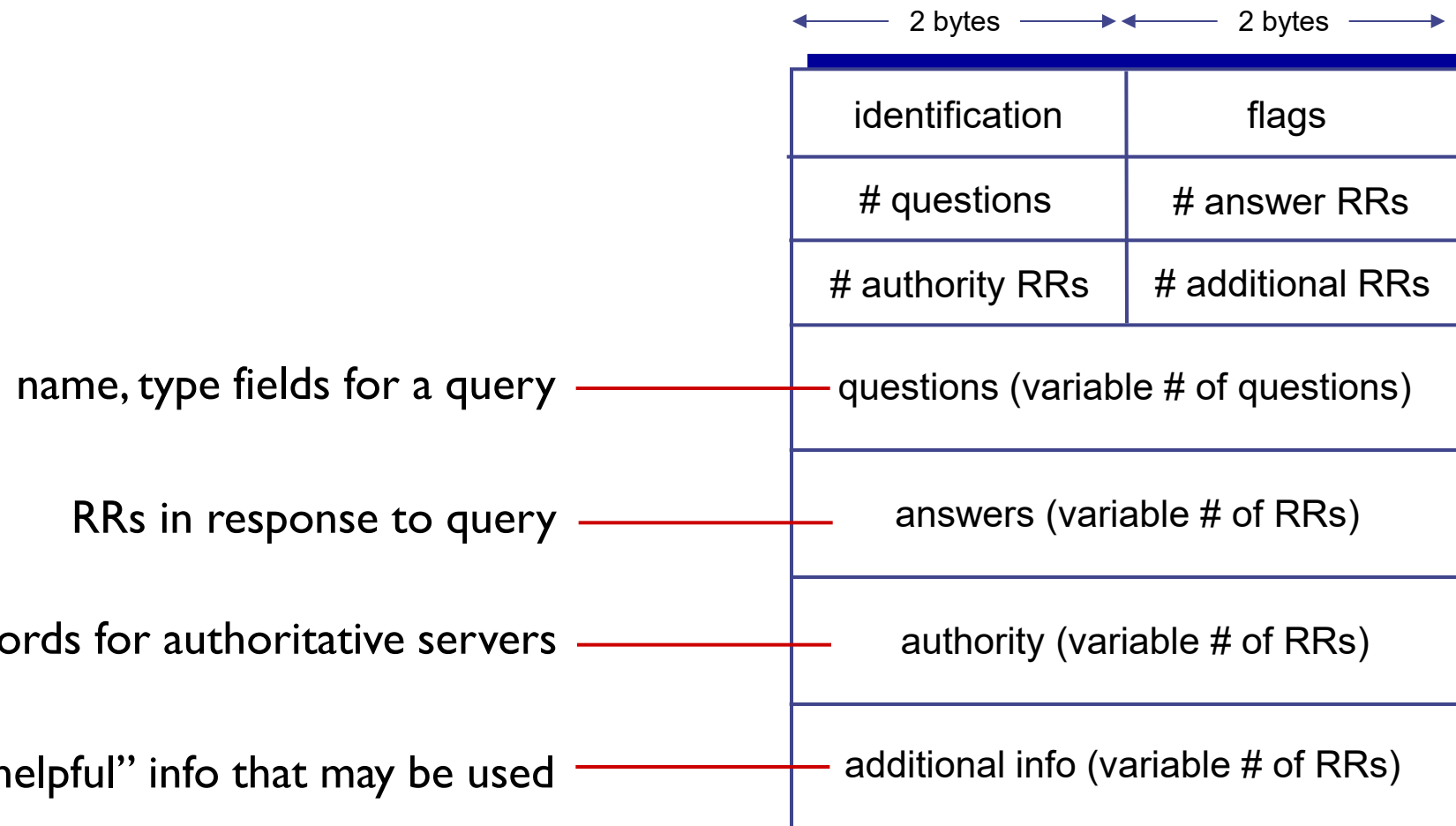
- **Query** and **reply** messages, both with same message format

Message header

- **identification**: 16-bit number
reply to query uses same
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol, messages



Inserting records into DNS

- Example: new startup “Network Utopia”
- Register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - Provide names, IP addresses of authoritative name server (primary and secondary)
 - Registrar inserts two RRs into .com TLD server:
`(networkutopia.com, dns1.networkutopia.com, NS)`
`(dns1.networkutopia.com, 212.212.212.1, A)`
- Create authoritative server type A record for www.networkutopia.com; type MX record for networkutopia.com
- In reality you can allow third party DNS providers to be your authoritative DNS server

Danish Registrars.

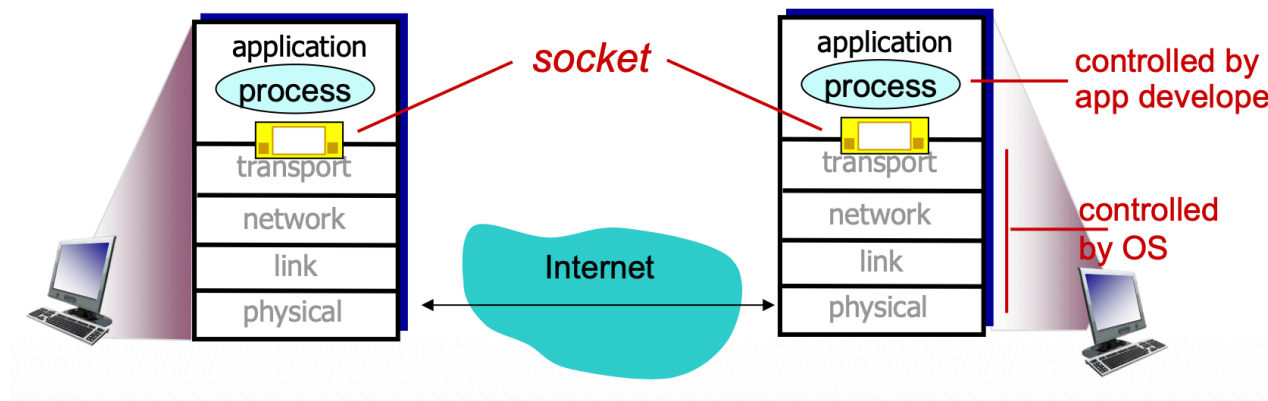
Registrar Name	IANA Number	Country/Territory	Public Contact
<u>Ascio Technologies, Inc. Danmark - Filial af Ascio technologies, Inc. USA</u> 	106	Denmark	Ashley La Bolle Tel: +49 2283296840 <u>nicrelations@ascio.com</u>
<u>One.com A/S</u> 	1462	Denmark	Rieke Poppe Tel: +45 46907100 <u>hostmaster@one.com</u>

Danish Registrars. Kilde: www.internic.net 26/9-2026

4.2 Socket Programming, revisited

Process to Process communication

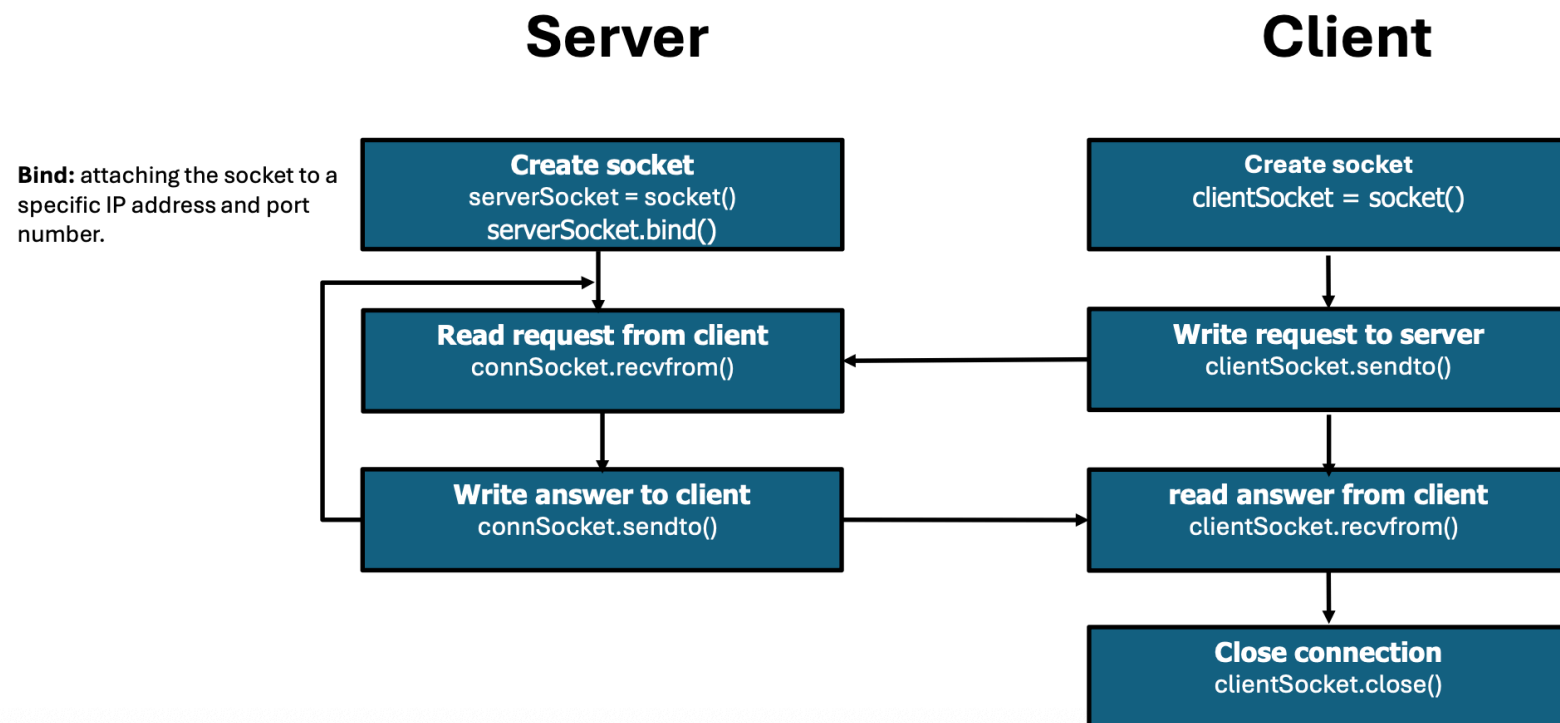
- At the application layer, we want process-to-process communication. (socket-to-socket)
- A process = a program that is running on a computer (like our web browser).
- Each process uses a socket to send and receive data.
- So when we say “process-to-process communication”, it really means:
- “Data is sent from one program’s socket on one computer to another program’s socket on another computer.”



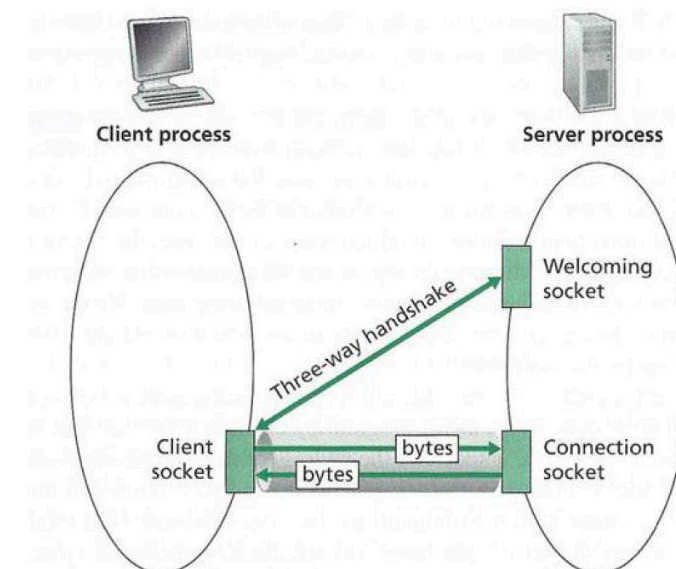
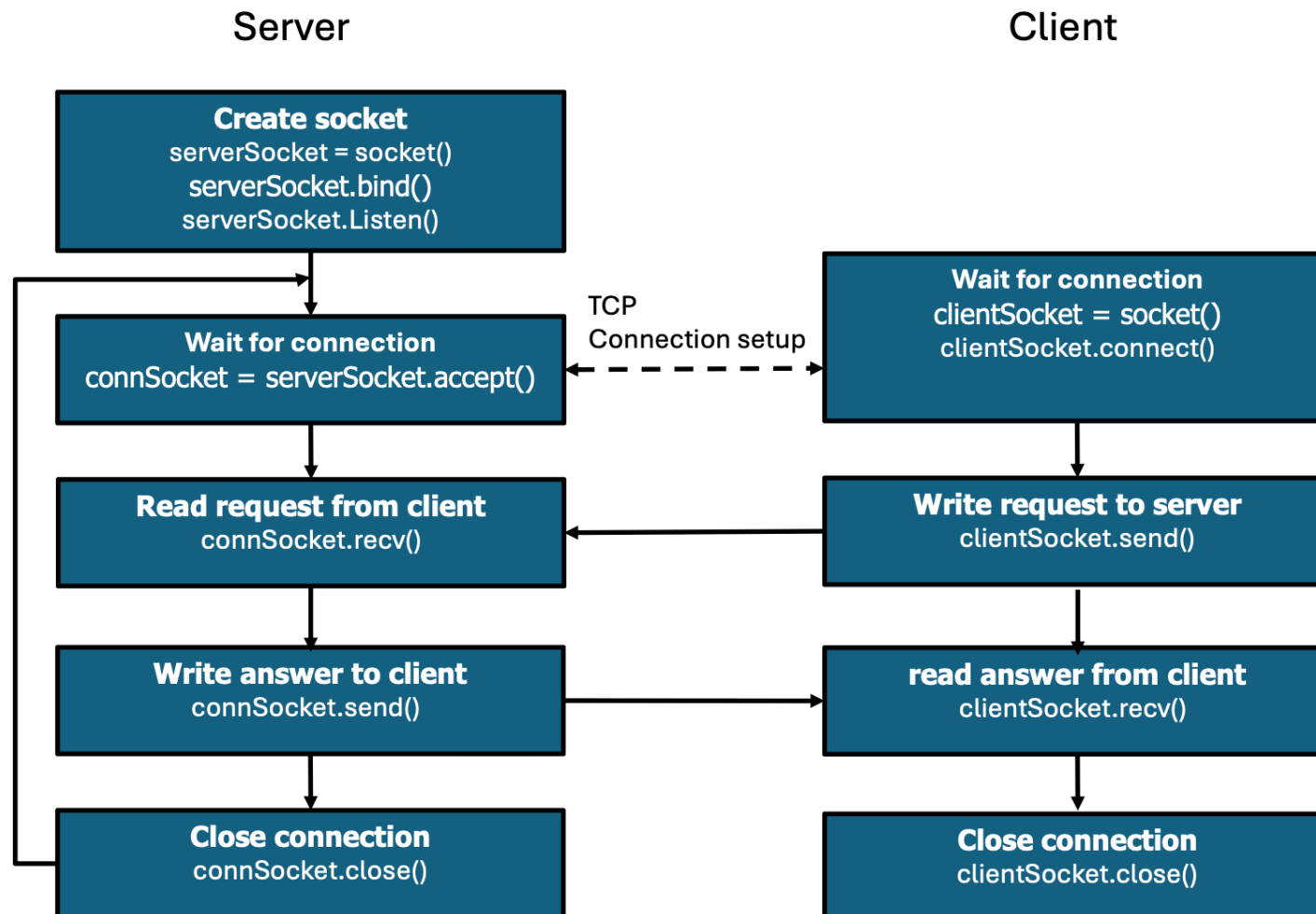
Client Server Program using Sockets

- Socket: an abstraction for processes at the application layer to connect through the network.
 - Needs to know:
 - UDP or TCP # select transport layer protocol
 - Source Address # for the IP-header
 - Source Port number # for the TCP/UDP header
 - Destination Address # for the IP-header
 - Destination Port number # for the TCP/UDP header

The client-server application using UDP



The client-server application using TCP



Today's Lab Work

Wireshark Lab

- Lab 3: DNS
 - nslookup
 - ipconfig
 - See: <https://public-dns.info/nameserver/dk.html> for public DNS servers
 - ns.sni.dk works

Python Labs

- Python Lab 1: Getting started with python –
 - See also: <https://www.python.org/about/gettingstarted/>
 - <https://docs.python.org/3/library/venv.html>
- Python Lab 2: Socket programming

Python is the supported programming language for exercises and is mandatory for related hand-in.

Next time

We move down the stack to the transport layer.
Read in chapter 3 (3.1-3.5) in the book (page 211-285).